

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: Total Marks: 20

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
```

```

<head>
<title>TestPage</title>
</head>
<body>
<div>
<p>HelloWorld
</div>
</body>
</html>

```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```

<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph

</body>
</html>

```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

I _G.Mohammad shareef(192311288) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:G.Mohammad shareef

RUBRICS FOR CALCULATION:

Criteria	Weigh tage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understandi ng of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some	Basic analysis with weak reasoning	No meaningful analysis provided

			justification		
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Answers

1. HTTP Response Analysis

1. Identify any incorrect or invalid fields in the header.

- **Content-Length: -150** is invalid because Content-Length cannot be negative.
- Other fields (**HTTP/1.1 200 OK**, **Date**, **Content-Type**, **Connection**) are valid.

2. Analyze the issue with Content-Length and explain its impact.

- Content-Length specifies the size of the response body in bytes.
- A negative value is invalid.
- This may cause browsers to reject the response or fail to load the page correctly.

3. Determine whether the status code matches the response correctness.

- Status code **200 OK** means the request was successful.
- Since the header contains an invalid Content-Length, the response is not completely correct.
- The status code does not match the invalid response.

4. Suggest corrections for the identified errors.

HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: 150
Connection: keep-alive

2. DOM Structure Analysis

1. Identify missing or improperly closed elements.

- Missing **</p>** tag.
- **<p>** is not properly closed before **</div>**.

2. Analyze how the browser will interpret this structure.

- The browser automatically closes the **<p>** element before closing the **<div>**.
- The page will still be displayed.

3. Determine the impact on rendering and layout.

- Most browsers render it correctly.

- Invalid HTML may cause unexpected layout issues in different browsers.

4. Suggest corrections to make the DOM valid.

```
<html>
<head>
<title>TestPage</title>
</head>
<body>
<div>
<p>Hello World</p>
</div>
</body>
</html>
```

3. GET vs POST in Login System

1. Interpret the differences in data transmission between GET and POST.

GET	POST
Data sent in URL	Data sent in request body
Visible to everyone	Not visible in URL
Less secure	More secure
Used for retrieving data	Used for sending sensitive data

2. Analyze which method is more suitable for login and why.

- **POST** is more suitable because credentials are sent in the request body and are not exposed in the URL.

3. Identify security risks in using GET for authentication.

- Username and password appear in the URL.
- Stored in browser history.
- Logged by servers.
- Can be shared accidentally.

4. Suggest best practices for secure login implementation.

- Use **POST** method.
- Use **HTTPS**.
- Encrypt passwords.
- Use secure session management.

- Validate user input.
-

4. HTML Code Inspection

1. Identify missing closing tags.

- Missing `</h1>`
- Missing `</p>`

2. Analyze issues with the `<img>` tag.

- `<img>` has no `alt` attribute.
- Adding `alt` improves accessibility.

3. Determine how browsers will handle these errors.

- Browsers automatically close missing tags.
- The page usually displays correctly, but the HTML is invalid.

4. Corrected HTML Code

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome</h1>
<p>This is a paragraph</p>

</body>
</html>
```

5. Browser–Server Communication Flow

1. Interpret each step in the communication flow.

1. User enters a URL.
2. DNS converts the domain name into an IP address.
3. Browser sends an HTTP request.
4. Server processes the request.
5. Server sends an HTTP response.

6. Browser renders the webpage.

2. Identify where delays can occur.

- DNS lookup.
- Network connection.
- Server processing.
- Downloading page resources (images, CSS, JavaScript).

3. Analyze the role of DNS in the process.

- DNS translates the website name into an IP address.
- Without DNS, the browser cannot locate the web server.

4. Suggest optimizations to improve performance.

- Use DNS caching.
- Enable browser caching.
- Compress files (Gzip/Brotli).
- Use a CDN.
- Optimize images and reduce file sizes.
- Minify CSS and JavaScript.